퍼셉트론 학습 알고리즘

오차 · 학습률 · 활성화 함수 · 경사하강법

오늘 배울 내용

01 **퍼셉트론 학습**

데이터셋, 구조, 오차, 학습률, w 업데이트

02 **활성화 함수**

왜 쓰는지, 선형과 비선형의 차이

03 **경사하강법**

기울기 방향, 수식 유도, step size, local minima

04 **코드 실습**

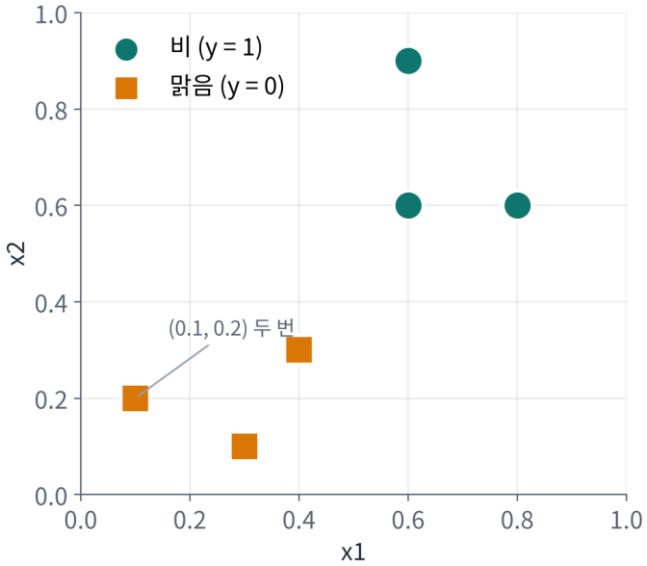
0.1초마다 학습 과정을 그리는 애니메이션

01 퍼셉트론 학습

틀린 만큼, 기여한 만큼, 조금씩 고친다

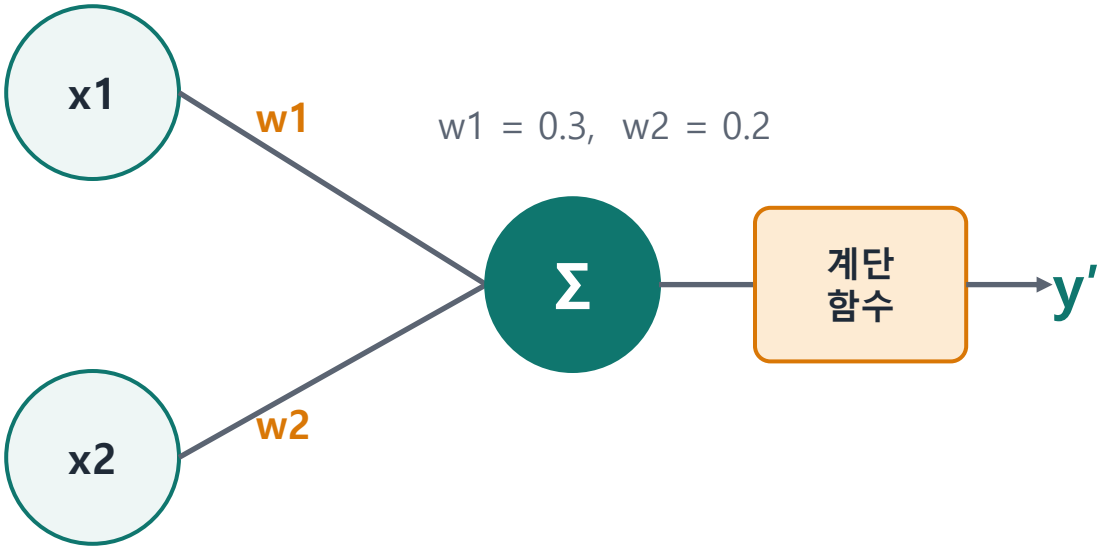
날씨를 숫자로 정리한 데이터셋

x1	x2	y (정답)
0.8	0.6	1 비
0.6	0.9	1 비
0.1	0.2	0 맑음
0.3	0.1	0 맑음
0.6	0.6	1 비
0.4	0.3	0 맑음
0.1	0.2	0 맑음



직선 하나로 두 그룹을 나눌 수 있는 데이터예요

퍼셉트론의 구조



연결강도(가중치) w1, w2

입력이 얼마나 중요한지 나타내는 값. 학습으로 찾아야 해요

가중합 Σ

x1·w1 + x2·w2. 입력 2개를 숫자 1개로 합쳐요

계단 함수

Σ가 0.5 이상이면 1, 아니면 0을 내보내요

첫 번째 데이터로 예측해 보기

입력 (0.8, 0.6), w1 = 0.3, w2 = 0.2

0.8 × 0.3 = 0.24

0.6 × 0.2 = 0.12

Σ = 0.36

0.36 < 0.5 → y′ = 0

1

정답 y

0

예측 y′

비 오는 날인데 맑다고 예측했어요. 틀렸으니 w를 고쳐야 해요.

오차 = 실제값 − 예측값

실제 y	예측 y′	오차	뜻	w를 어떻게?
1	1	0	맞힘	그대로
0	0	0	맞힘	그대로
1	0	**+1**	Σ가 너무 작았음	**w를 키움**
0	1	**−1**	Σ가 너무 컸음	**w를 줄임**

이번 예제는 초기 w가 작아서 오차가 항상 +1만 나와요. 그래서 학습 내내 w가 커지기만 해요.

오차의 부호 = 고칠 방향

출력이 0 또는 1뿐이라 오차도 +1, 0, −1 세 가지예요

오차가 0이면 w가 안 바뀜

"틀렸을 때만 고친다"가 식 안에 자동으로 들어 있어요

퍼셉트론의 학습 규칙

새 w = 현 w + 입력값 × 오차 × 학습률

오차

어느 방향으로?

+1이면 키우고, –1이면 줄이고, 0이면 그대로

입력값

누구를 더 많이?

입력이 클수록 틀린 결과에 더 크게 기여했다고 보고 더 많이 고쳐요

학습률

얼마나 많이?

아주 작은 수를 곱해서 w를 조금씩 바꿔요 (예: 0.01)

첫 번째 업데이트 계산하기

w1 $0.3 + 0.8 \times 1 \times 0.01 \rightarrow \mathbf{0.308}$ x1 = 0.8

w2 $0.2 + 0.6 \times 1 \times 0.01 \rightarrow \mathbf{0.206}$ x2 = 0.6

x1(0.8)이 x2(0.6)보다 커서 w1이 더 많이 늘었어요. 입력값이 클수록 더 많이 고친다는 뜻이에요.

첫 번째 바퀴(epoch 1)에서 w가 바뀌는 과정

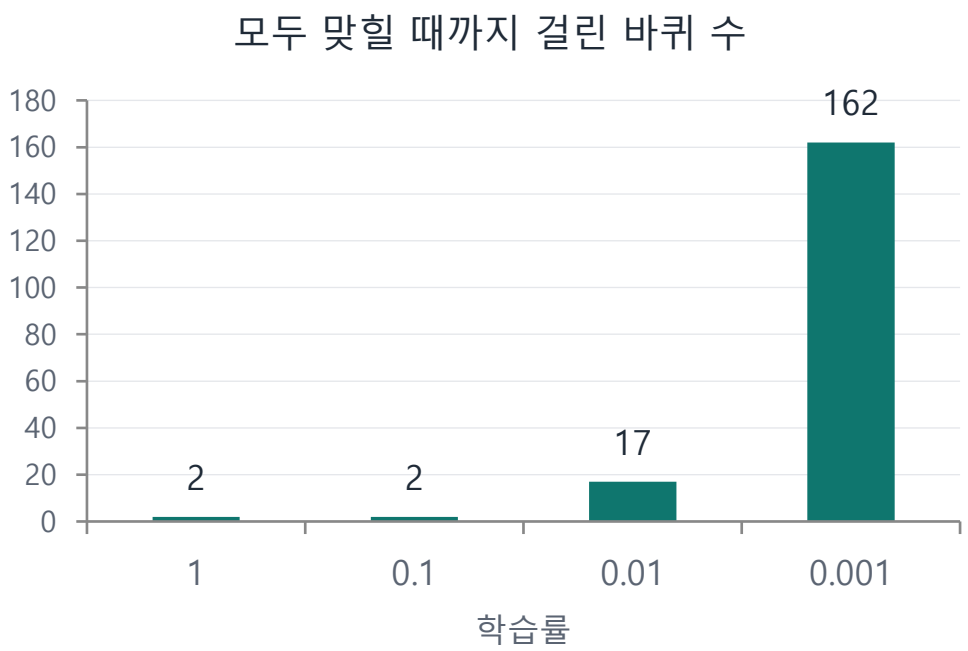
#	x1	x2	Σ	y′	y	오차	새 w1	새 w2
1	0.8	0.6	0.360	0	1	+1	0.308	0.206
2	0.6	0.9	0.370	0	1	+1	0.314	0.215
3	0.1	0.2	0.074	0	0	0	0.314	0.215
4	0.3	0.1	0.116	0	0	0	0.314	0.215
5	0.6	0.6	0.317	0	1	+1	0.320	0.221
6	0.4	0.3	0.194	0	0	0	0.320	0.221
7	0.1	0.2	0.076	0	0	0	0.320	0.221

틀린 데이터

3개

1, 2, 5번만 w를 고쳤어요. 나머지는 맞혀서 그대로예요.

학습률에 따라 달라지는 학습 속도



작으면: 안정적이지만 느림

0.001은 0.01보다 약 10배 많이 반복해야 했어요

크면: 한 번에 크게 튐

학습률 1이면 첫 업데이트에서 w1이 0.3 → 1.1. 쉬운 데이터라 빨리 끝났지만, 복잡한 데이터에서는 정답 근처를 오가며 수렴하지 못할 수 있어요

17번째 바퀴에서 모두 맞힘

17 바퀴

학습률 0.01일 때 걸린 epoch

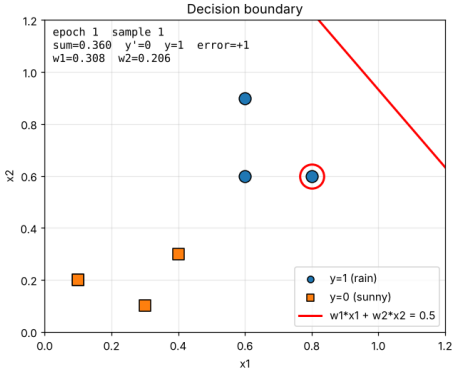
0.3 → 0.466

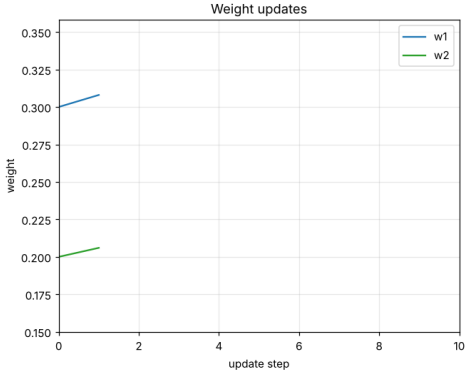
w1의 변화

0.2 → 0.371

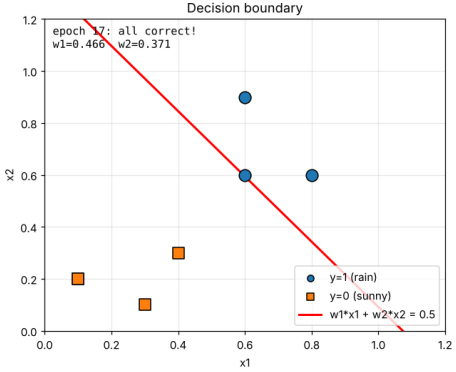
w2의 변화

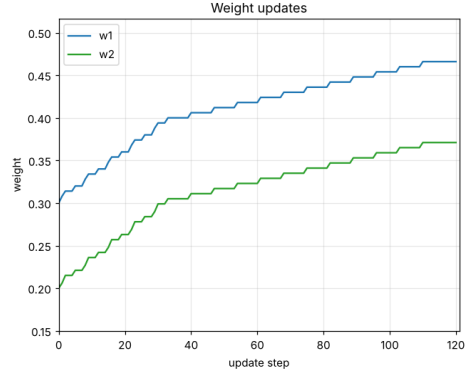
시작





끝





02 활성화 함수는 왜 쓸까

핵심은 값을 줄이는 게 아니라 '비선형성'

활성화 함수가 없으면 층을 쌓아도 직선 하나

1층

$$h = 2x + 1$$

→

2층

$$y = 3h + 4$$

→

정리하면

$$y = 6x + 7$$

곱하기와 더하기만 반복하면 항상 wx + b 꼴로 정리돼요. 층을 100개 쌓아도 1층과 능력이 같아요.

층 사이에 휘어지거나 꺾인 함수를 넣으면 식이 하나로 합쳐지지 않아서, 층을 쌓을수록 복잡한 모양을 표현할 수 있어요

활성화 함수를 쓰는 세 가지 이유

1 **비선형성**

직선으로 나눌 수 없는 경계(XOR, 원형, 나선형)나 곡선 관계를 표현할 수 있어요. 가장 중요한 이유예요.

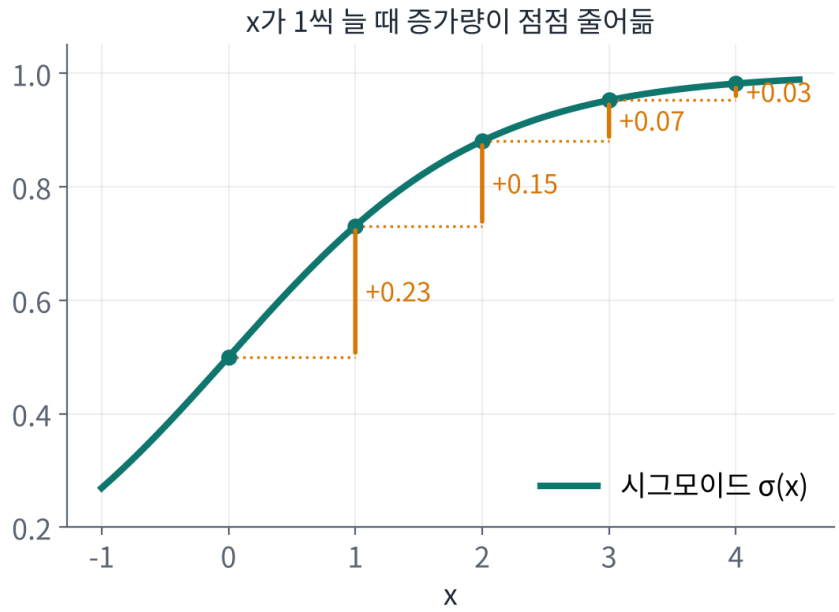
2 **출력 모양 맞추기**

출력층에서 시그모이드는 0~1 확률로, 소프트맥스는 합이 1인 확률로 바꿔 줘요.

3 **미분 가능**

역전파는 미분을 곱해 가는 방식이라 미분할 수 있어야 해요. 계단 함수는 미분이 안 돼서 여러 층을 학습시킬 수 없었어요.

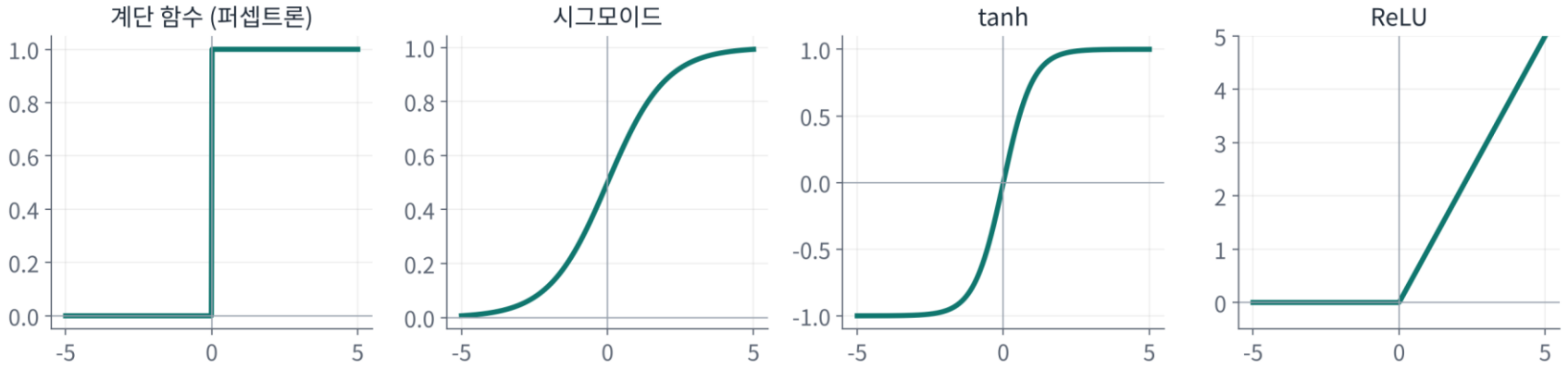
시그모이드는 왜 비선형일까



x	y = 2x	증가량	σ(x)	증가량
0	0	–	0.50	–
1	2	+2	0.73	+0.23
2	4	+2	0.88	+0.15
3	6	+2	0.95	+0.07

- 직선: 기울기가 어디서나 같아요
- 시그모이드: 위치마다 기울기가 달라요
- ReLU: 0에서 꺾여 있어서 비선형

자주 쓰는 활성화 함수



계단 함수

퍼셉트론. 미분 불가

시그모이드

0~1. 이진 분류 출력층

tanh

−1~1. RNN 은닉층

ReLU

max(0, x). 은닉층 기본값

03 경사하강법

기울기의 반대 방향으로 조금씩 내려가기

안개 낀 산을 내려오는 방법

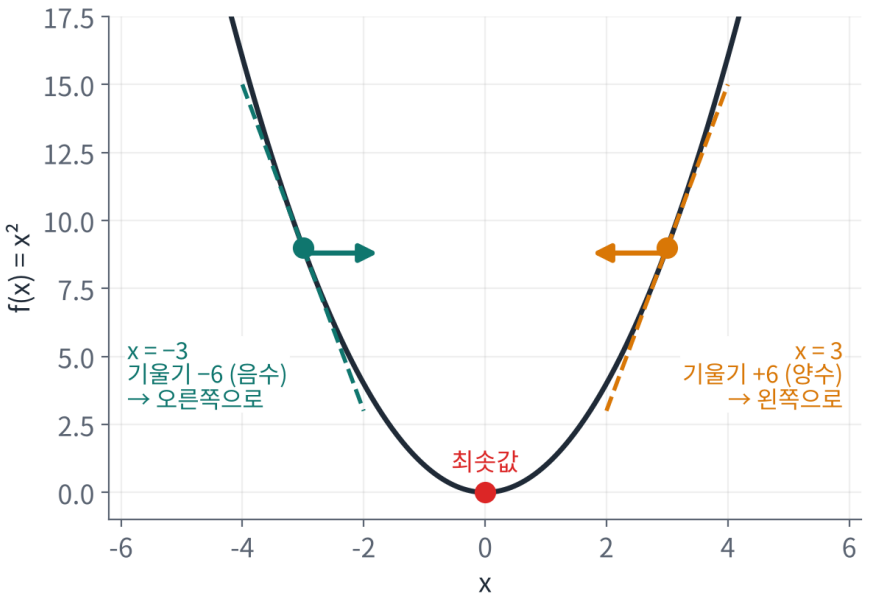
앞이 보이지 않으면, 발로 더듬어 가장 낮아지는 방향으로 한 걸음씩 내딛으면 돼요.

기울기가 양수 (+)

x가 커질수록 f가 커짐 → x를 줄여야 함 (왼쪽)

기울기가 음수 (–)

x가 커질수록 f가 작아짐 → x를 키워야 함 (오른쪽)



왜 f′(x) = 0을 직접 풀지 않을까

1

복잡한 함수

신경망의 loss는 매우 복잡하고 비선형이라 f′(x) = 0의 해를 식으로 구하기 어려워요

2

구현이 쉬움

기울기를 구하고 조금 이동하는 것을 반복하기만 하면 돼서 컴퓨터로 구현하기 쉬워요

3

데이터가 많을 때

데이터가 아주 많으면 한 번에 푸는 것보다 반복해서 조금씩 고치는 쪽이 계산이 효율적이에요

그래서 "지금 위치의 기울기를 보고 한 걸음"을 반복하는 방법을 써요.

경사하강법 수식 만들기

1 **기울기 부호의 반대로 이동**

$x_{i+1} = x_i - \text{이동거리} \times \text{기울기의 부호}$

양수를 빼면 왼쪽, 음수를 빼면 오른쪽으로 가요

2 **기울기 크기도 쓰기**

멀면 크게, 가까우면 작게

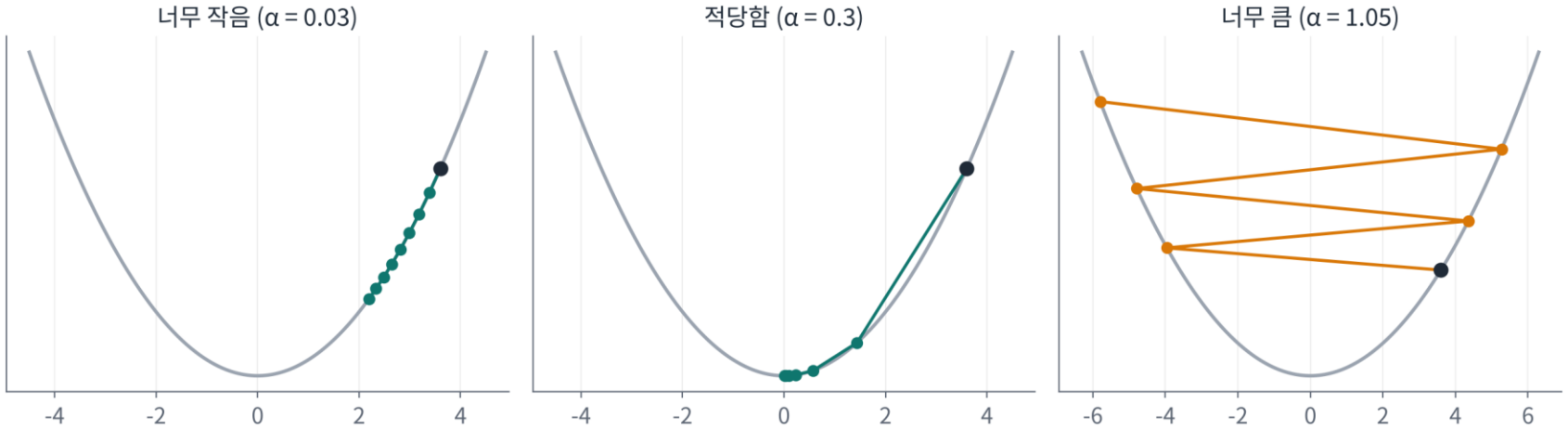
최솟값에 가까울수록 기울기가 작아지니까 이동량에 그대로 곱해요

3 **최종 수식**

$x_{i+1} = x_i - \alpha \cdot f'(x_i)$

α = step size(학습률). 신경망에서는 w ← w − lr × ∂L/∂w

step size(학습률)는 적당하게



너무 작음

방향은 맞지만 너무 느려서 정해진 시간 안에 도착하지 못해요

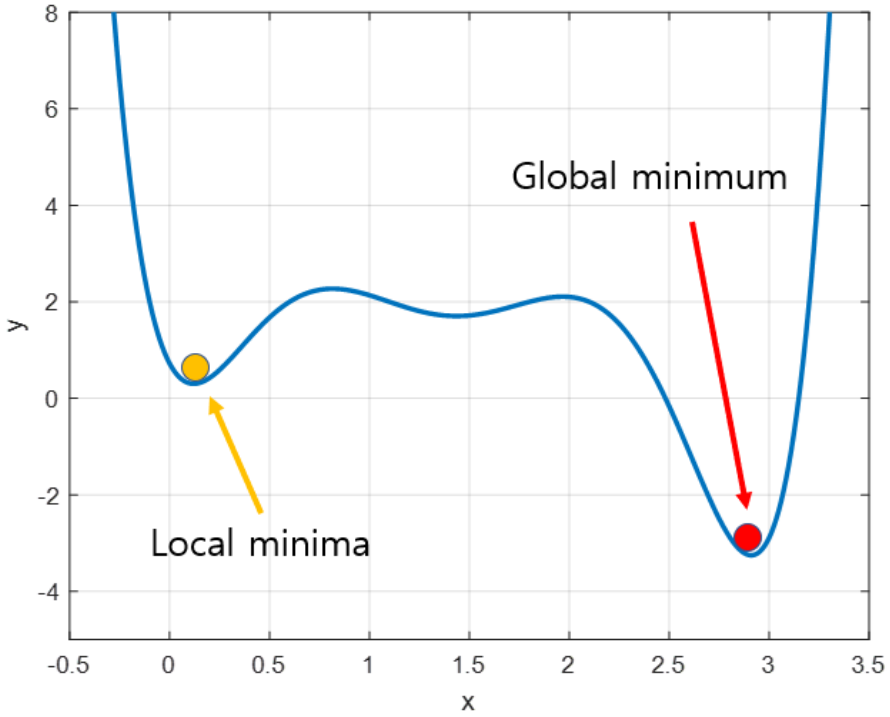
적당함

몇 번 만에 최솟값에 빠르게 도착해요

너무 큼

최솟값을 지나쳐 반대편으로 튀고, 점점 멀어지며 발산해요

local minima 문제



무엇이 문제?

시작점이 랜덤이라 운이 나쁘면 가장 낮은 곳이 아닌 작은 웅덩이에 빠져 나오지 못해요

어떻게 줄이나?

- 모멘텀: 이전에 움직이던 방향의 관성으로 넘어가기
- 여러 시작점에서 학습해 보기

퍼셉트론 학습 규칙 vs 경사하강법

	퍼셉트론 학습 규칙	경사하강법
업데이트 식	$w \leftarrow w + lr \times (y - y') \times x$	$w \leftarrow w - lr \times \partial L/\partial w$
오차	–1, 0, 1 중 하나	틀린 정도가 연속값으로 나옴
활성화 함수	계단 함수 (미분 불가)	시그모이드, ReLU 등 (미분 가능)
언제 고치나	틀렸을 때만	매번, 틀린 정도만큼
여러 층	불가능	**역전파로 가능**

식 모양은 거의 같아요.
계단 함수를 미분 가능한 함수로 바꾸고 loss를 미분해 기울기를 구하도록 발전시킨 게 경사하강법이에요.

04 코드 실습

0.1초마다 학습 과정을 그려 보기

학습 핵심 코드

```
w1, w2, lr = 0.3, 0.2, 0.01

def step(s):
    return 1 if s >= 0.5 else 0

for epoch in range(1, 31):
    wrong = 0
    for (x1, x2), y in zip(X, Y):
        s = x1 * w1 + x2 * w2
        y_pred = step(s)
        error = y - y_pred
        w1 = w1 + x1 * error * lr
        w2 = w2 + x2 * error * lr
        wrong += abs(error)
    if wrong == 0:
        break
```

가중합

s = x1·w1 + x2·w2

활성화 함수

0.5 이상이면 1

오차

$y - y'$

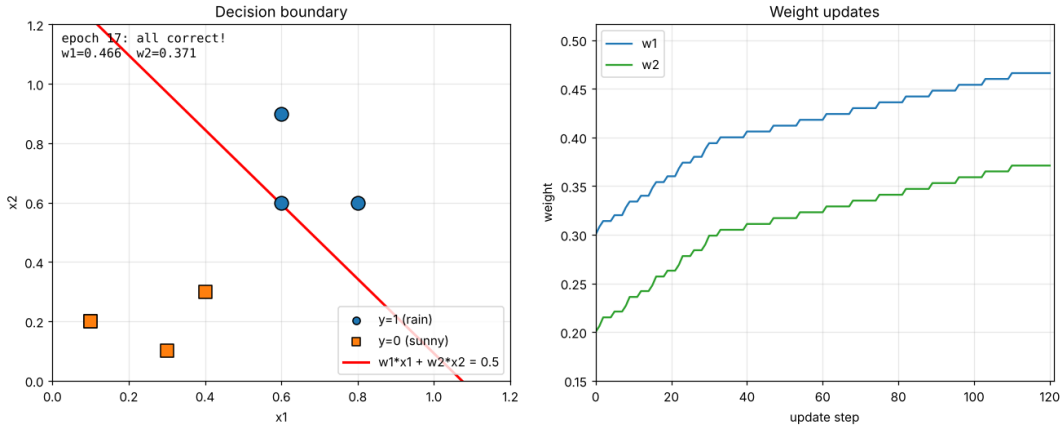
학습 규칙

w + 입력값 × 오차 × lr

종료 조건

한 바퀴 동안 모두 맞히면 멈춤

0.1초 단위 애니메이션



- 왼쪽: 데이터 점과 결정 경계(빨간 직선)
- 오른쪽: 업데이트마다 w1, w2 값 변화
- FuncAnimation(interval=100) → 0.1초마다 데이터 1개씩 학습하고 다시 그려요

직접 바꿔 보기

- lr = 1 → 직선이 크게 점프
- lr = 0.001 → 아주 천천히 (MAX_EPOCH 200 이상)
- w1, w2 = 0.8, 0.8 → 오차 −1, w가 줄어듦
- INTERVAL_MS = 300 → 더 천천히 보기

정리

01 **퍼셉트론 학습**

새 w = 현 w + 입력값 × 오차 × 학습률. 틀린 만큼, 기여한 만큼, 조금씩

02 **활성화 함수**

비선형성을 넣어 층을 쌓는 의미를 만들고, 미분 가능해야 역전파가 돼요

03 **경사하강법**

$x \leftarrow x - \alpha \cdot f'(x)$. 기울기 반대로, 적당한 step size로, local minima 주의

참고 자료

- 퍼셉트론 예제 데이터와 흐름: 신박AI,「[Neural Network 2] 인공신경망과 퍼셉트론 2」(YouTube)
- 경사하강법 설명: 공돌이의 수학정리노트,「경사하강법(gradient descent)」 angeloyeo.github.io/2020/08/16/gradient_descent.html
- 모든 그림은 이 자료를 위해 새로 그렸어요. 학습률 비교, epoch 추적 수치는 직접 계산한 결과예요.